

Dynamic Collaborative Scheduling System for New Energy Logistics Fleets Based on Graph Algorithms and Reinforcement Learning

Zepeng Lin
AI Class2
202430690300

Guanyu Xin
AI Class2
202464870413

Yuxuan Xue
AI Class2
202430841574

<https://github.com/oplisty/Data-Structure-HW>

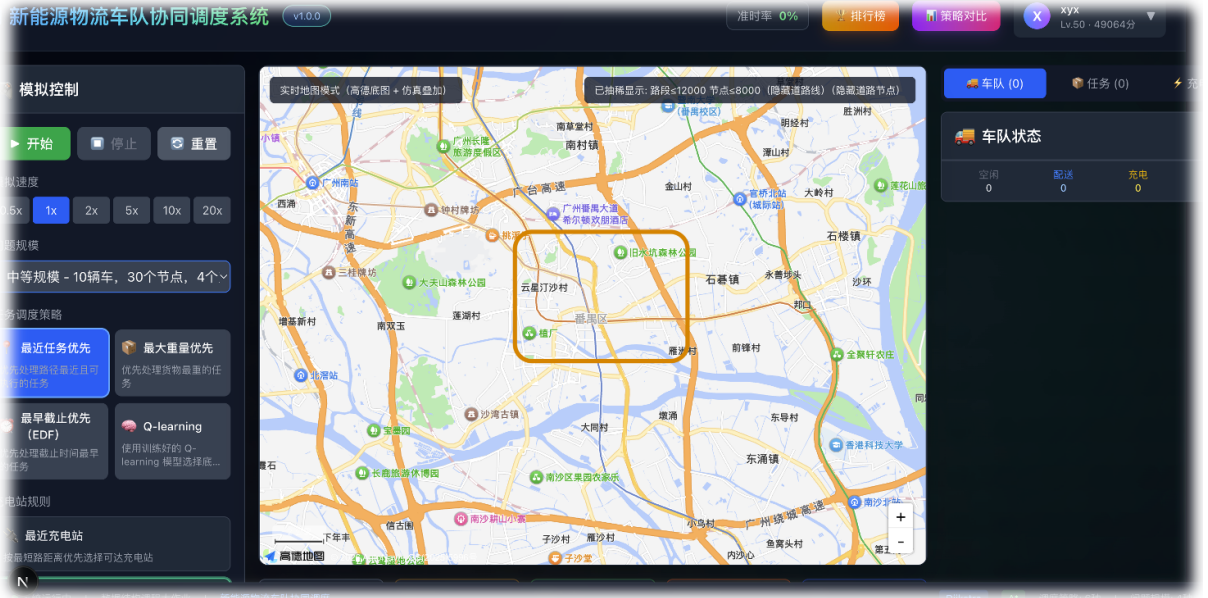


Figure 1. Visualization interface of the dynamic collaborative scheduling system for new energy logistics fleets.

Abstract

Urban new energy logistics scheduling is more complex than finding the shortest route. In practice, vehicles have to make decisions while tasks keep arriving, battery and load capacity are limited, charging takes time, stations may have queues, and unfinished tasks may miss their deadlines. To study this process, we built a multi-vehicle scheduling simulator and used it to compare different dispatching strategies.

The road network is represented as a weighted graph. Dijkstra and A* are used for shortest-path and feasibility queries, while RRT is kept as an extensible path-planning module. For scheduling, we implement Nearest, EDF, Heaviest, a Q-learning based rule selector, and an offline MILP

reference for small-scale full-information cases. The simulator is connected with a backend service and a visualization frontend, so the scheduling process can be executed, observed, and compared.

Experiments on controlled random graph scenarios show that Nearest is a strong fixed baseline, while Q-learning can learn useful rule-selection behavior but is less stable in medium-scale settings. As an extension, we also build a processed Guangzhou Panyu OSM road-network scenario to test the same simulator on a real urban map. The focus is a runnable, visible, and comparable scheduling platform rather than a single algorithm.

1. Introduction

With the growth of e-commerce and instant delivery, urban logistics systems are required to finish more tasks within shorter time windows. New energy logistics vehicles are attractive because of their low emissions and low operating cost, but they also bring additional constraints. A vehicle may not have enough battery to serve a distant task, charging takes time, charging stations may be occupied, and tasks may expire if they are not completed before their deadlines.

The goal of this project is not to solve one static route. Instead, the project focuses on a repeated online decision process: tasks appear over time, vehicles choose feasible actions, and the simulator records how different strategies perform under the same environment.

To make the simulation closer to a real scheduling process, we model more than vehicle positions and path length. Each vehicle keeps its battery level, load capacity, current status, and energy consumption rate. Each task has a release time, location, cargo weight, deadline, and completion state. Charging stations are treated as limited resources with occupied chargers and waiting queues. These details make the scheduler consider not only which task is close, but also whether the vehicle can reach it, finish it before the deadline, and recover battery when needed.

The implemented work can be grouped into three parts:

- **Environment modeling:** graph road network, depot, task nodes, vehicles, battery states, load limits, task deadlines, charging queues, and random scenario generation for different scales.
- **Scheduling algorithms:** Dijkstra and A* are used for path queries, RRT is kept as an extensible planning module, and Nearest/EDF/Heaviest are implemented as fixed baselines. On top of these baselines, we add two advanced modules: a Q-learning based hyper-heuristic for adaptive rule selection, and an MILP offline solver that provides a global-planning reference for small-scale full-information cases.
- **System interface:** backend simulation control, event recording, strategy switching, Q-learning training/inference support, and frontend visualization for vehicles, tasks, charging stations, movement traces, queues, and result comparison.

2. System Framework

Figure 2 shows how the system runs. Instead of testing each algorithm separately, we put all strategies into the same simulation loop. At each step, the simulator releases tasks, updates vehicle states, checks path and battery feasibility, assigns tasks, executes movement or charging, and records statistics. The main modules behind this loop are graph-based environment data, path planning and scheduling algorithms, the simulation engine, backend services, and frontend

tend visualization.

2.1. Overall Running Process

The complete simulation loop can be summarized in three stages:

1. task release and vehicle state update;
2. path/battery feasibility checking and scheduling decision;
3. movement, service, charging, and statistics update.

At each time step, the simulator releases new tasks, updates vehicle states, checks path and battery feasibility, assigns tasks to available vehicles, and then executes movement, service, return, or charging. After execution, it updates completed tasks, expired tasks, score, travel distance, and charging queue statistics.

2.2. Graph-based Environment Modeling

The data resource layer provides a unified representation of the environment for the whole system and serves as the foundation for subsequent path planning, task scheduling, and state simulation. To satisfy the course requirement of using a graph structure for road representation and routing, this paper first models the urban delivery scenario as a weighted directed graph

$$G = (V, E, W),$$

where the node set V represents road intersections, depots, candidate task locations, and charging stations; the edge set $E \subseteq V \times V$ represents road connectivity; and the weight function $W : E \rightarrow \mathbb{R}^+$ describes the distance or travel cost of each edge. Based on this representation, the entire logistics environment can be mapped into a discrete graph space, which provides a computable foundation for shortest-path search, reachability analysis, and energy consumption evaluation.

The simulator supports two types of map data. For controlled experiments, we use randomly generated graph scenarios with different numbers of nodes, vehicles, tasks, and charging stations, so that scheduling strategies can be compared under the same settings. As an extension, the codebase also includes a processed Guangzhou Panyu road-network scenario based on Guangdong Province OSM data. The Panyu road topology is converted into a weighted graph with road points and intersections as nodes, road segments as weighted edges, and charging stations as resource nodes. In the processed metadata, the Panyu map contains 131,276 nodes, 142,593 edges, and 57 charging stations. This allows the same path-planning, scheduling, backend, and frontend pipeline to run on a real urban road network.

In the implementation, the system denotes the depot as node $v_d \in V$, the set of charging stations as

$$S = \{s_1, s_2, \dots, s_m\} \subseteq V,$$

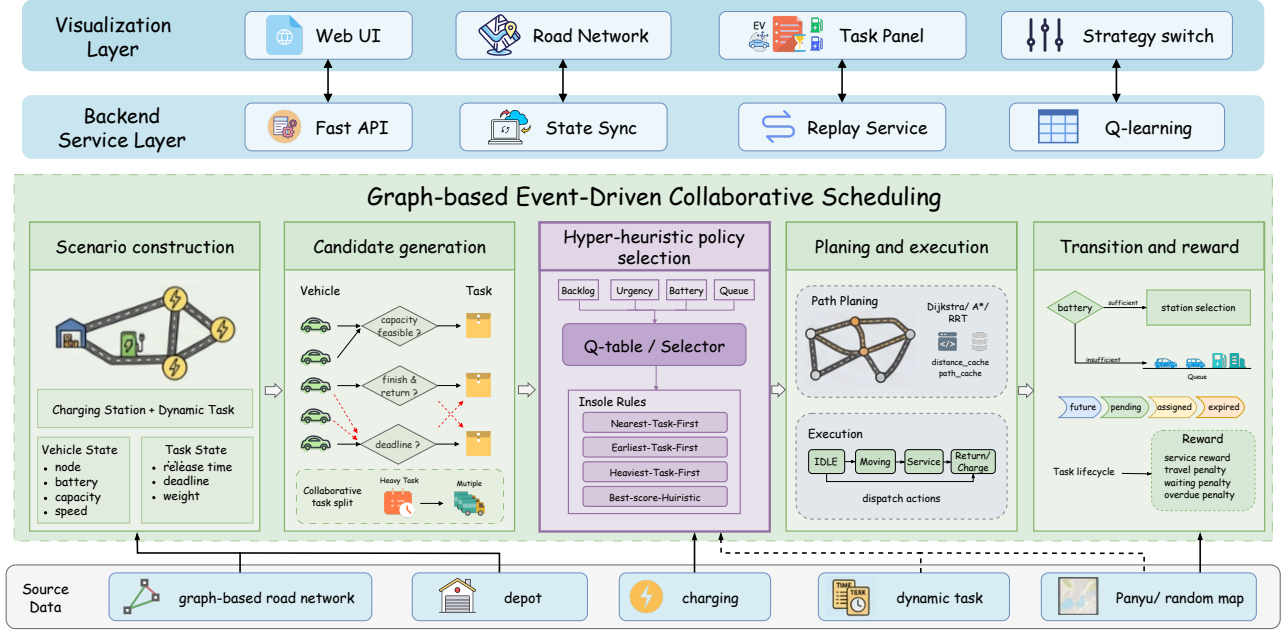


Figure 2. Overall framework of the dynamic collaborative scheduling system for new energy logistics fleets. The figure illustrates the relationships among the visualization frontend, backend service layer, simulation engine, scheduling strategies, path planning algorithms, and data resources.

and generates dynamic tasks from the candidate task node set $V_T \subseteq V$. For any task τ_i , it can be represented as the following four-tuple:

$$\tau_i = (r_i, v_i, w_i, d_i),$$

where r_i is the release time of the task, $v_i \in V_T$ is the task location, w_i is the cargo weight, and d_i is the deadline. This representation allows each task to include not only spatial information but also explicit time constraints, thereby supporting online scheduling decisions in dynamic arrival scenarios.

Furthermore, the data resource layer maintains the static parameter sets of vehicles and stations. For example, for vehicle k , its state parameters can be defined as

$$\mathbf{x}_k = (b_k^{\max}, b_k, c_k, v_k^{\text{cur}}, \eta_k),$$

where $b_k^{\max}$ and b_k denote the maximum battery level and current battery level, respectively; c_k is the load capacity limit; v_k^{cur} is the current node location; and η_k is the energy consumption per unit distance. For charging station s_j , the system records attributes such as the number of chargers, queue length, and charging rate. In this way, the data resource layer provides not only the static topology of the road network but also a unified description of the parameters of tasks, vehicles, and charging facilities.

2.3. Algorithm Layer

After the data resource layer completes the formal representation of the environment, the algorithm layer is further responsible for generating executable routing and scheduling decisions on the graph-based road network $G = (V, E, W)$. In this paper, this layer is divided into two closely coupled basic modules: the path planning module and the task scheduling module.

2.3.1. Path Planning

For any vehicle k and target node $v \in V$, the goal of the path planning module is to find a feasible path $P_k(v_k^{\text{cur}}, v)$ on graph G from the current location v_k^{cur} to the target node v . Let the total cost of a path P be

$$C(P) = \sum_{e \in P} W(e),$$

then the shortest path problem can be written as

$$P_k^*(v_k^{\text{cur}}, v) = \arg \min_{P \in \mathcal{P}(v_k^{\text{cur}}, v)} C(P),$$

where $\mathcal{P}(v_k^{\text{cur}}, v)$ denotes the set of all feasible paths from v_k^{cur} to v .

Based on this unified objective, the system implements three path planning baselines.

- **Dijkstra:** A classic shortest-path algorithm for weighted graphs. It directly minimizes the total path cost $C(P)$ and

can stably return the global optimal path. Therefore, it is used as the standard shortest-path baseline in the system.

- **A*:** This method introduces a heuristic evaluation function $h(u, v)$ on top of Dijkstra and uses

$$f(n) = g(n) + h(n)$$

as the search priority, where $g(n)$ is the known accumulated cost from the start node to the current node n , and $h(n)$ is the heuristic estimated cost from n to the target node. This method maintains good search quality while reducing unnecessary node expansions, thereby improving the efficiency of each query.

- **RRT:** A sampling-based search method. It does not directly guarantee a strict shortest path in the graph sense, but explores a large state space through random tree expansion. This provides an interface for future research under more complex path planning settings.

In the new energy delivery scenario, path planning is not only used for travel from the current node to a task node, but also needs to support feasibility checking for returning to the depot after task completion and visiting a charging station when the battery is low. Therefore, for a task $\tau_i = (r_i, v_i, w_i, d_i)$, the system not only computes $C(P_k^*(v_k^{\text{cur}}, v_i))$, but also further evaluates whether the vehicle satisfies the basic energy consumption constraint. Let η_k denote the energy consumption per unit distance of vehicle k . Then the energy consumption along path P can be approximately written as

$$E_k(P) = \eta_k C(P).$$

Only when the current battery level b_k is sufficient to support the planned trip is the path regarded as executable.

2.3.2. Task Scheduling

After feasible paths are obtained, the scheduling module further determines which task should be assigned to which vehicle. For the currently visible task set $\mathcal{T}_t$ and the current state $\mathbf{x}_k$ of vehicle k , a baseline scheduling strategy essentially defines a priority score function $\phi_k(\tau_i)$ over all candidate tasks and selects

$$\tau_k^* = \arg \max_{\tau_i \in \mathcal{T}_t} \phi_k(\tau_i)$$

as the task with the highest priority at the current time. The difference between different baselines lies in how the score function $\phi_k(\tau_i)$ is defined.

- **Nearest-task-first strategy:** This strategy uses the minimum path cost as the main criterion. Its policy can be written as

$$\tau_k^* = \arg \min_{\tau_i \in \mathcal{T}_t} C(P_k^*(v_k^{\text{cur}}, v_i)).$$

- **Maximum-weight-first strategy:** This strategy focuses on giving priority to high-load tasks. For a task $\tau_i = (r_i, v_i, w_i, d_i)$, its policy can be written as

$$\tau_k^* = \arg \max_{\tau_i \in \mathcal{T}_t, w_i \leq c_k} w_i.$$

- **Earliest-deadline-first strategy:** This strategy takes task urgency as the main consideration, and can be expressed as

$$\tau_k^* = \arg \min_{\tau_i \in \mathcal{T}_t} d_i.$$

Table 1. Roles of algorithms and strategies in the system.

Algorithm / Strategy	Role in the system
Dijkstra	shortest-path computation and stable routing baseline
A*	goal-directed path search with heuristic acceleration
RRT	extensible path-planning interface for future scenarios
Nearest	selects the nearest task to reduce distance and energy cost
EDF	selects the task with the earliest deadline
Heaviest	selects the task with the largest cargo weight
Q-learning	selects a suitable dispatching rule under the current state
MILP	provides a small-scale offline full-information reference

Beyond the above baselines, we further implement an online reinforcement learning policy selection module based on Q-learning, as well as an offline global optimization module based on MILP under full information. Q-learning is used as a hyper-heuristic selector rather than an end-to-end route generator. Its state includes coarse information such as task backlog, deadline urgency, average battery level, idle vehicle ratio, and charging queue load. Its action is the choice of a low-level dispatching rule such as Nearest, EDF, Heaviest, or a score-based rule. The reward is related to task completion, travel cost, overdue penalty, and charging or waiting cost. These two advanced modules correspond to two different methodological paradigms: online decision-making based on interactive learning and offline solving based on exact optimization, as summarized in Algorithm 1 and Algorithm 2.

2.4. Simulation Layer

The simulation engine layer is responsible for transforming the static environment defined in the data resource layer and the scheduling actions produced by the algorithm layer into an executable time-based process. It is the core component that enables dynamic closed-loop simulation in the whole system. Unlike methods that focus only on a single path query or a local dispatch decision, this layer explicitly models state variables that evolve over time, including task release, vehicle movement, battery consumption, charging queues, and task completion. In this way, the new energy logistics scheduling problem can be continuously solved and evaluated along a unified timeline.

In implementation, the system adopts a discrete-time update mechanism. Let the global environment state at time t

Algorithm 1: Q-learning-based Policy Selection

Require: training episodes M , learning rate α , discount factor γ , initial exploration rate ϵ , decay factor ρ , discrete state space S , action space A

Ensure: learned Q-table Q

```

1: Initialize  $Q(s, a) = 0$ , for all  $(s, a) \in S \times A$ 
2: for episode = 1 to  $M$  do
3:   Reset the initial state  $s_0$ 
4:   while the episode is not terminated do
5:     Generate a random number  $k \in [0, 1]$ 
6:     choose a random action  $a_t$  is random if  $k < \epsilon$  else  $a_t = \arg \max_a Q(s_t, a)$ 
7:     Execute action  $a_t$ 
8:     Observe reward  $r_t$ , next state  $s_{t+1}$ , and the termination flag
9:     Update the Q-table:
10:     $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$ 
11:     $s_t \leftarrow s_{t+1}$ 
12:   end while
13:    $\epsilon \leftarrow \max(\epsilon_{\min}, \rho \cdot \epsilon)$ 
14: end for
15: return  $Q$ 

```

Algorithm 2: Offline MILP-based Global Planning Given the full-information task set $\mathcal{T}$, vehicle set $\mathcal{V}$, expanded node set $\mathcal{N}$, and arc cost matrix C , the offline planner solves the following mixed-integer linear program:

$$\min \sum_{v \in \mathcal{V}} \sum_{(i,j) \in \mathcal{A}} C_{ij} x_{vij} + \lambda_1 \sum_{v \in \mathcal{V}} \sum_{n \in \mathcal{T}} \text{tardy}_{vn} + \lambda_2 T_{\max}. \quad (1)$$

The optimization is subject to

$$\sum_{v \in \mathcal{V}} \text{visit}_{vn} = 1, \quad \forall n \in \mathcal{T}, \quad (2)$$

$$\sum_{j: (s,j) \in \mathcal{A}} x_{vsj} = \text{use}_v, \quad \sum_{i: (i,t) \in \mathcal{A}} x_{vit} = \text{use}_v, \quad \forall v \in \mathcal{V}, \quad (3)$$

$$\sum_{i: (i,n) \in \mathcal{A}} x_{vin} = \sum_{j: (n,j) \in \mathcal{A}} x_{vnj} = \text{visit}_{vn}, \quad \forall v \in \mathcal{V}, n \in \mathcal{N} \setminus \{s, t\}, \quad (4)$$

$$\text{load}_{vj} \leq \text{load}_{vi} - w_j + M_\ell(1 - x_{vij}), \quad \forall v \in \mathcal{V}, (i, j) \in \mathcal{A}, \quad (5)$$

$$\text{soc}_{vj} \leq \text{soc}_{vi} + \text{chg}_{vi} - e_{vij} + M_b(1 - x_{vij}), \quad \forall v \in \mathcal{V}, (i, j) \in \mathcal{A}, \quad (6)$$

$$\text{arr}_{vj} \geq \text{arr}_{vi} + \text{srv}_i + \text{chgT}_{vi} + \tau_{ij} - M_t(1 - x_{vij}), \quad \forall v \in \mathcal{V}, (i, j) \in \mathcal{A}, \quad (7)$$

$$\text{arr}_{vn} \geq r_n, \quad \text{tardy}_{vn} \geq \text{arr}_{vn} - d_n, \quad \forall v \in \mathcal{V}, n \in \mathcal{T}. \quad (8)$$

Solve the above MILP using Gurobi/CPLEX, decode the semantic route, and replay the resulting plan in the simulation engine for comparison.

be denoted by $\mathcal{S}_t$, and let the joint action generated by the algorithm layer be denoted by $\mathcal{A}_t$. Then, the simulation engine can be formalized as a state transition process:

$$\mathcal{S}_{t+1} = \Phi(\mathcal{S}_t, \mathcal{A}_t),$$

where $\Phi(\cdot)$ denotes the environment transition operator jointly defined by task updates, vehicle movement, energy consumption updates, charging scheduling, and statistical

recording. More specifically, at each time step, the system first adds newly released tasks into the pending-task set according to their release times and removes tasks that have already passed their deadlines. It then calls the scheduler to generate dispatch actions based on the current vehicle states and the candidate task set. After that, the system executes path movement and task service, while updating vehicle positions, remaining battery levels, load states, and task com-

pletion status. Finally, it processes charging-station events such as queuing, charger occupation, and battery replenishment, and synchronously refreshes system-level statistics.

The key role of this layer is to extend path planning and task scheduling from a *static selection problem* to a *dynamic evolution problem under resource constraints*. For example, when vehicle k travels along path P , its battery level decreases continuously according to the energy consumption model defined earlier, namely $E_k(P) = \eta_k C(P)$. When b_k is no longer sufficient to support the planned trip, the simulation engine triggers charging-station selection and queueing logic, and updates the waiting state under limited station capacity. Similarly, for dynamically released tasks, the simulation engine determines not only when a task enters the assignable set, but also whether it becomes a penalty item because of deadline violation. As a result, metrics such as task reward, vehicle utilization, path length, and charging load can all be computed consistently within a unified simulation framework.

2.5. Frontend and Backend

The frontend layer and the backend layer do not directly participate in path planning or scheduling optimization itself. Instead, they are responsible for organizing the states, events, and statistical results inside the simulation engine into external representations that are accessible, replayable, and analyzable. In this way, the whole system is equipped with a complete experimental loop and interactive capability.

In implementation, the backend is responsible for unified state packaging and service exposure. The system uses FastAPI and WebSocket to establish a persistent communication channel, and projects the environment state $\mathcal{S}_t$ at time t into frontend-consumable data objects, such as vehicle positions, task states, charging-station load, accumulated score, and event logs. At the same time, the backend also receives control commands from external users and converts them into invocation requests for the simulation process or algorithm modules. In addition to standard controls such as start, pause, resume, and stop, this layer also provides unified interfaces for Q-learning training and inference, offline MILP solving, and result export, so that the experimental process can be scheduled and reused within the same service framework. In other words, this layer essentially provides a mapping

$$\Psi : \mathcal{S}_t \mapsto \mathcal{O}_t,$$

where $\mathcal{O}_t$ denotes the structured observation result for external interfaces, thereby converting complex low-level states into standardized service outputs.

On this basis, the frontend visualization module further transforms $\mathcal{O}_t$ into graphical expressions that can be easily understood by users. Specifically, the interface continu-

ously displays the spatial distribution of the road network, depots, task nodes, charging stations, and vehicles, while synchronously updating key indicators such as task completion status, vehicle utilization, path cost, charging queue length, and system score. More importantly, this layer does not only serve as a static display component. It also provides a unified interaction entry for policy switching, offline replay, training triggering, and result comparison. As a result, the multi-source information originally scattered across simulation logs, scheduling outputs, and statistical results is reorganized into a visual workflow for analysis and presentation.

3. Experiments and Results

3.1. Experiment Setup

For the main quantitative comparison, all strategies are tested on the same controlled random graph settings, task sequences, vehicle configurations, and evaluation metrics. The experiments focus on three questions: which fixed heuristic is the most stable baseline, whether Q-learning can learn useful rule-selection policies, and how charging-station selection affects scheduling performance.

3.1.1. Environment Setup

The environment is built on a graph-based road network with dynamic tasks, limited vehicles, battery constraints, load constraints, charging stations, and task deadlines. We test three scenario scales:

- **Small:** 5 vehicles, 30 dynamic tasks, 2 charging stations, 25 road nodes, map size 30×30 , and simulation horizon 180.
- **Medium:** 10 vehicles, 100 dynamic tasks, 4 charging stations, 60 road nodes, map size 50×50 , and simulation horizon 300.
- **Large:** 20 vehicles, 300 dynamic tasks, 8 charging stations, 120 road nodes, map size 80×80 , and simulation horizon 480.

We also compare two charging-station selection rules: `optimal_station`, which considers both distance and station load, and `nearest_station`, which always selects the nearest station.

3.1.2. Evaluation Metrics

We report task effectiveness, operating cost, and learning-stage metrics. Task effectiveness is measured by completed tasks, expired tasks, and final score. Operating cost is measured by total distance, simulation steps, makespan for offline planning, and charging burden. For Q-learning, we additionally report total training reward and evaluation score mean, which reflects the performance of the greedy policy after training.

3.2. Experiment Results

3.2.1. Baseline Comparison

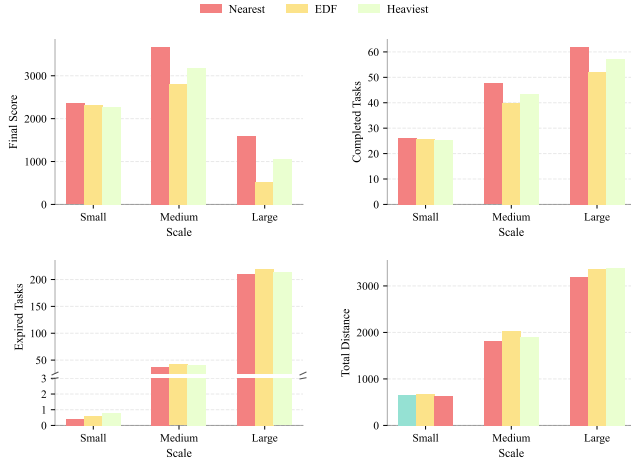


Figure 3. Main comparison of heuristic baselines across three problem scales.

Figure 3 compares Nearest, EDF, and Heaviest under different scenario scales. Since the total number of tasks differs across scales, completed tasks should be interpreted together with final score, expired tasks, and travel distance.

On the small scenario, the three heuristics are almost tied. Nearest obtains the highest average final score of **2354.10**, with **26.0** completed tasks and **0.4** expired tasks. EDF and Heaviest follow with scores of **2309.03** and **2271.05**. The gap is small because the task pressure is low and most vehicles can still find feasible tasks quickly.

The medium scenario separates the methods more clearly. Nearest remains the strongest baseline, with a final score of **3662.73**, **47.6** completed tasks, and **36.6** expired tasks. Heaviest ranks second, while EDF produces the weakest result among the three. A likely reason is that EDF focuses only on urgency and may send vehicles to distant tasks, which increases travel cost, battery consumption, and later deadline risk.

The large scenario is the most difficult one. All fixed rules suffer from more expired tasks, but Nearest still achieves the best score among fixed heuristics, followed by Heaviest and EDF. This suggests that distance-aware dispatching is a strong baseline in our simulator: shorter paths reduce energy consumption and leave more time for later tasks.

3.2.2. Comparison with Q-learning

Figure 4 compares the Q-learning hyper-heuristic with the strongest fixed baseline. In this experiment, Q-learning is evaluated as a rule-selection policy: it does not directly generate routes, but selects a low-level dispatching rule according to the current system state.

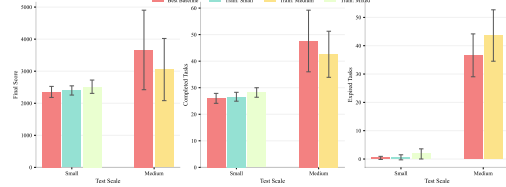


Figure 4. Overall comparison between Q-learning and the best baseline.

On small-scale evaluation, Q-HH-Small achieves a final score of **2399.09**, slightly higher than the best fixed baseline, while maintaining a similar task completion level. Q-HH-Mixed obtains a higher final score of **2515.96** and completes **28.2** tasks on average, but it also increases expired tasks to **1.8**. This suggests that mixed-scale training can improve task completion and score, but may also introduce more aggressive dispatching behavior.

In the medium-scale evaluation, Q-HH-Medium obtains a final score of **3050.90**, with **42.6** completed tasks and **43.6** expired tasks. This is lower than the Nearest baseline under the same scale. Therefore, the current tabular Q-learning policy is not consistently better than a strong fixed heuristic in more complex environments. The result is reasonable because medium-scale scenarios have denser task arrivals, larger state variation, and stronger coupling between task dispatching and charging decisions.

The training logs still show that Q-HH-Medium once reaches a best evaluation score of **4651.62**. This means Q-learning can discover high-quality rule-selection behavior, but the learned policy is not stable enough across evaluation seeds. Therefore, Q-learning is better viewed as a promising adaptive strategy rather than a fully dominant method in the current implementation.

3.2.3. Training Convergence



Figure 5. Convergence curves of Q-learning under different training scales.

Figure 5 shows that Q-learning improves policy quality during training, but its stability depends on scenario scale. The small-scale model is the most stable and reaches a best evaluation score of **2550.60** at episode **155**. The medium model reaches a higher best score of **4651.62** at episode **230**, but later evaluation performance decreases. The mixed-scale model also reaches a strong best score of **4455.95**, while its smoothed performance fluctuates because the training distribution alternates across different scales.

These results indicate that tabular Q-learning is feasible for adaptive rule selection, but its coarse state representation limits generalization in larger scenarios. Future work can improve this part by using richer state features, function approximation, or deep reinforcement learning methods such as DQN or PPO.

3.2.4. Charging Strategy Ablation

Table 2. Charging strategy ablation results on the medium-scale setting.

Method / strategy	Final Score	Completed	Expired	Avg. Queue
Fixed baseline + optimal station	–	47.6	36.6	0.018
Fixed baseline + nearest station	–	47.6	36.6	0.018
Q-learning + optimal station	2935.14	34.6	11.8	–
Q-learning + nearest station	2836.04	33.6	12.0	–

Table 2 compares `optimal_station` and `nearest_station` in the medium-scale setting. For the fixed Nearest scheduler, the two charging strategies produce almost identical results: both complete **47.6** tasks, produce **36.6** expired tasks, and have an average queue load of only **0.018**. This suggests that charging congestion is not the main bottleneck in the current fixed-baseline runs.

For Q-learning, optimal station selection performs slightly better than nearest station selection. It obtains a final score of **2935.14**, compared with **2836.04** under nearest station selection. It also completes slightly more tasks and produces fewer expired tasks. This shows that charging decisions can interact with adaptive scheduling policies, although the effect is still limited under the current queue-load setting.

The experiments lead to three main findings. First, Nearest is a strong fixed baseline because it reduces both travel cost and energy risk. Second, Q-learning can learn useful rule-selection policies, but the current tabular version is not stable enough in medium-scale scenarios. Third, charging strategy design matters more when it interacts with adaptive scheduling, while its effect is small when station queues are light.

4. Conclusion

We deliver a runnable scheduling and visualization system for energy-aware logistics fleets. Instead of focusing on a

single algorithm, the implementation connects graph-based routing, online task scheduling, battery and charging simulation, backend control, and frontend visualization into one workflow.

The experiments show that Nearest is a strong fixed baseline in the current settings, because shorter routes also reduce energy consumption and deadline risk. Q-learning can learn useful rule-selection behavior, especially in small and mixed-scale settings, but its medium-scale performance is still unstable. The charging ablation also suggests that station selection matters more when it interacts with adaptive scheduling.

The project also leaves room for data and scenario extension. While the main experiments use controlled random graph scenarios for fair comparison, we also build a Guangzhou Panyu OSM-based road-network extension. This shows that the same simulator can be reused on both synthetic graphs and real urban road topology.

Future work can build on this extensibility. The learning module can be improved with richer state features, function approximation, or deep reinforcement learning methods. The real-map scenario can also be extended with more realistic task demand, vehicle parameters, traffic conditions, and charging-station usage data, so that the platform moves closer to practical logistics simulation.

5. Project Reflection

At the beginning, we treated the problem mainly as a combination of shortest-path search and task assignment. After implementing the simulator, we realized that the difficult part was the interaction among modules. A nearby task may still be a poor choice when the vehicle has low battery, the deadline is tight, or charging stations are crowded. Therefore, we paid more attention to state updates, feasibility checks, battery recovery, and scheduling decisions over time.

The weighted graph became the core data structure of the project. It supported not only shortest-path queries, but also reachability checking, energy estimation, charging-station selection, and strategy comparison. This became clearer when we used both random testing graphs and the processed Panyu OSM map. The Q-learning and MILP modules also helped us understand the difference between adaptive online decisions and offline global planning, while the frontend made the process easier to observe and debug. Overall, this project showed us that a strong course project depends not only on adding algorithms, but also on connecting data structures, simulation logic, experiments, and visualization into a working system.

Dynamic Scheduling System for New Energy Logistics Fleets Based on Graph Algorithms and Reinforcement Learning

Supplementary Material

6. Evaluation Metrics

This supplementary section gives the mathematical definitions of the evaluation metrics used in the experiments. The main paper explains the meaning and comparison results of these metrics, while the supplementary material keeps the formulas for reproducibility.

6.1. Task Effectiveness Metrics

Final Score. For each completed task τ_i , the task score is defined as

$$\text{Score}(\tau_i) = \max \left(0, R_0 - \alpha \cdot \min \left(\frac{\ell_i}{D_{\text{ref}}}, 1 \right) - P_{\text{late}} \cdot \mathbb{I}(t_i^{\text{finish}} > D_i) \right).$$

Here, R_0 is the base reward, ℓ_i is the service travel distance of task i , D_{ref} is the reference distance for normalization, α is the distance-cost weight, P_{late} is the late-completion penalty, and D_i is the deadline of task i . In our implementation, we use

$$R_0 = 100, \quad \alpha = 30, \quad P_{\text{late}} = 60.$$

This scoring function does not directly penalize task waiting time. Instead, it considers whether the task is completed, how long the service travel distance is after normalization, and whether the task is completed after its deadline.

The total final score is computed as

$$\text{FinalScore} = \sum_{i \in \mathcal{T}_{\text{completed}}} \text{Score}(\tau_i) - P_{\text{fail}} \cdot |\mathcal{T}_{\text{expired}}| + S_{\text{partial}}.$$

Here, $P_{\text{fail}} = 60$ by default, $\mathcal{T}_{\text{completed}}$ is the set of completed tasks, $\mathcal{T}_{\text{expired}}$ is the set of unfinished expired tasks, and S_{partial} denotes optional partial credit for collaborative tasks.

Completed Tasks. Completed Tasks denotes the number of tasks completed by the end of the simulation:

$$\text{CompletedTasks} = |\mathcal{T}_{\text{completed}}|.$$

Expired Tasks. Expired Tasks denotes the number of tasks that pass their deadlines and remain unfinished by the end of the simulation:

$$\text{ExpiredTasks} = |\mathcal{T}_{\text{expired}}|.$$

Total Tardiness. Total Tardiness is mainly used for offline MILP result analysis and is defined as

$$\text{TotalTardiness} = \sum_n \max(0, \text{arr}_n - D_n),$$

where arr_n denotes the arrival time at task n , and D_n is its deadline.

6.2. Operating Cost Metrics

Total Distance. Total Distance is the sum of travel distances of all vehicles:

$$\text{TotalDistance} = \sum_{v \in \mathcal{V}} \text{dist}_v.$$

Steps. Steps denotes the number of discrete simulation time steps from the initial time to the end of the online simulation.

Makespan. Makespan is used for offline MILP result analysis and denotes the maximum completion time after all vehicles finish service and return to the depot:

$$\text{Makespan} = \max_{v \in \mathcal{V}} T_v^{\text{finish}}.$$

Charging Burden. Charging Burden is measured using two statistics: the number of charging triggers and the average queue load. The number of charging triggers measures how often vehicles enter the charging process, while the average queue load measures congestion caused by competition for charging resources.

6.3. Training-stage Metrics

Total Reward. Total Reward is the sum of immediate rewards over all decision steps in a single Q-learning training episode:

$$\text{TotalReward} = \sum_t r_t.$$

Eval Score Mean. Eval Score Mean is the mean final score obtained by applying the greedy policy over several evaluation episodes after each training round:

$$\text{EvalScoreMean} = \frac{1}{K} \sum_{k=1}^K \text{FinalScore}^{(k)}.$$

This metric measures the stable performance of the learned policy and avoids relying only on a single training trajectory.